

---

# Python Basics (2)

...

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan

---

---

# Agenda

- Python Basics
  - In-class exercise 1 feedback
  - In-class exercise 2 introduction

---

# Python Basics: A Structured Introduction

---

---

# What's in the reading materials

## Week 2. Intro to Programming

- Arithmetic and variables
  - Functions
  - Data types
  - Conditions and conditional statements
  - Intro to lists
-

---

# What's in the reading materials

## Week 3. Python

- Hello, Python
  - Functions and getting help
  - Booleans and conditionals
  - Lists
  - Loops and list comprehensions
  - Strings and dictionaries
  - Working with external libraries
-

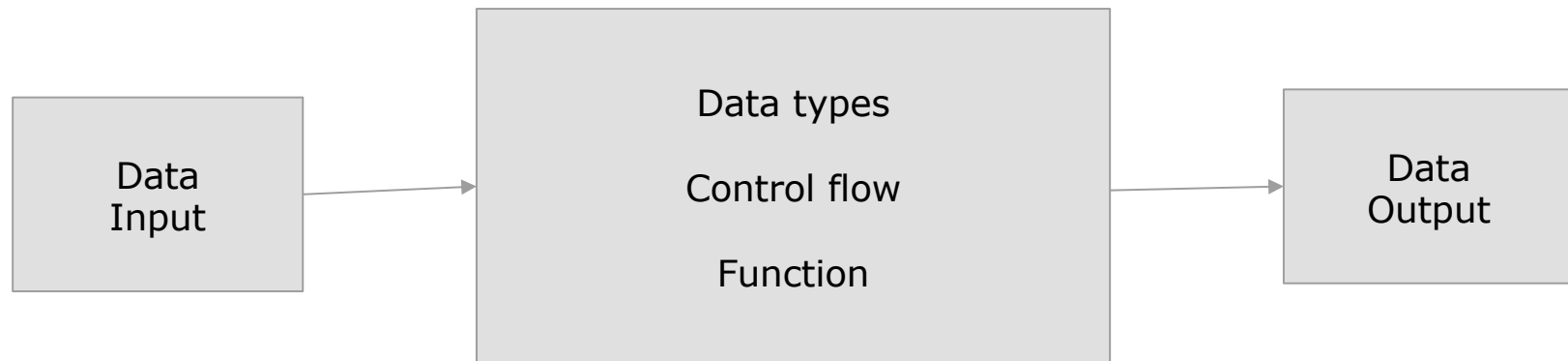
Our job today: Get a structured understanding of the skills

Simply put, we learn them because they serve one of the following steps in our data science workflow

**input** data,  
**process** the data, and  
**output** data.

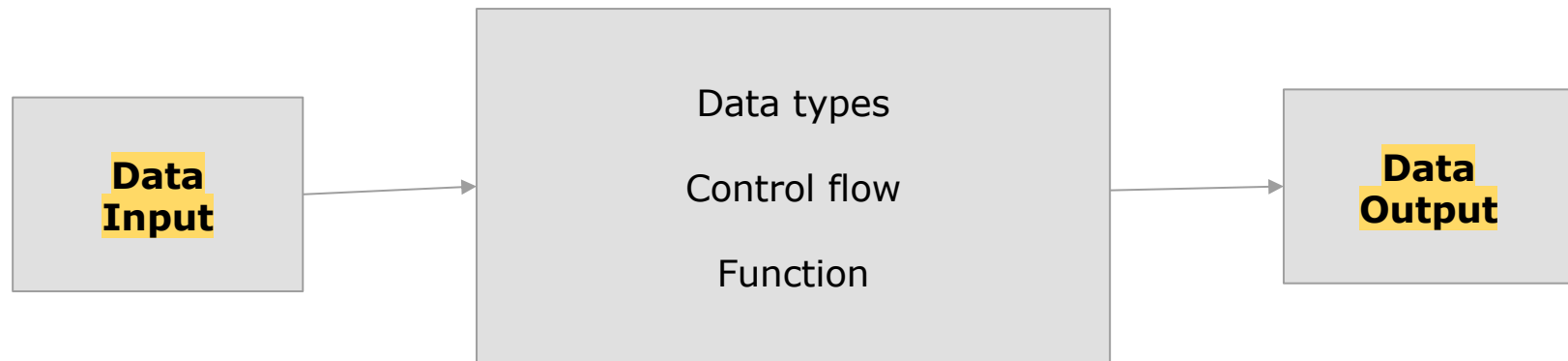
# Map skills into our data science workflow

- Data Input
- Data processing
  - Data types
  - Control flow
  - Function
- Data Output



# Input

- **Data Input**
- Data processing
  - Data types
  - Control flow
  - Function
- **Data Output**





---

# Input...

... is about getting data into Python



# Input

- Your data analysis usually starts with data input
- Input without assignment (get data **through** Python)
  - Ex.  $1 + 1$
- Input with assignment (get data **into** Python)
  - Ex.  $x = 1$

**Assignment** is a crucial operator for data input.

---

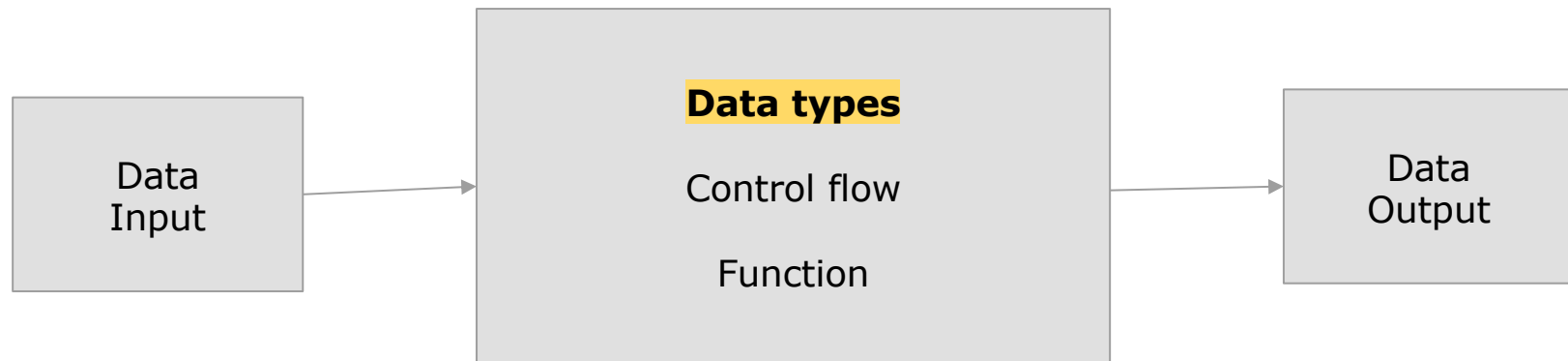
# Output

Output results of your data analysis

- Route 1: Print in the notebook
  - Export to external files (to be discussed in future lectures)
-

# Data Types and Operators

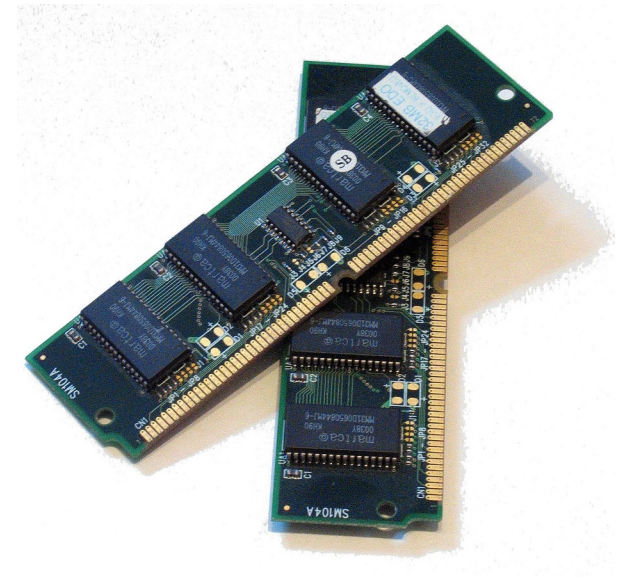
- Data Input
- Data processing
  - **Data types**
  - Control flow
  - Function
- Data Output



# Data Types and Operators...

... are about how data are “understood” by Python

- **Data types:** Python categorize data into different types  
→ Put them on the hardware in different ways
- **Different data types**
  - Store different types of data (of course...)
  - Allow different types of *operations*



---

**Three big categories of data types**

**Numbers, Sequences, Set, Mapping**

---

---

## Data Types for Numbers

- **Integers:** 0, 1, 2, 3, ...
  - **Real:** 1.2, 2.3, 4.57, 9.97
  - **Booleans:** True, False
-

# Operators for Numbers

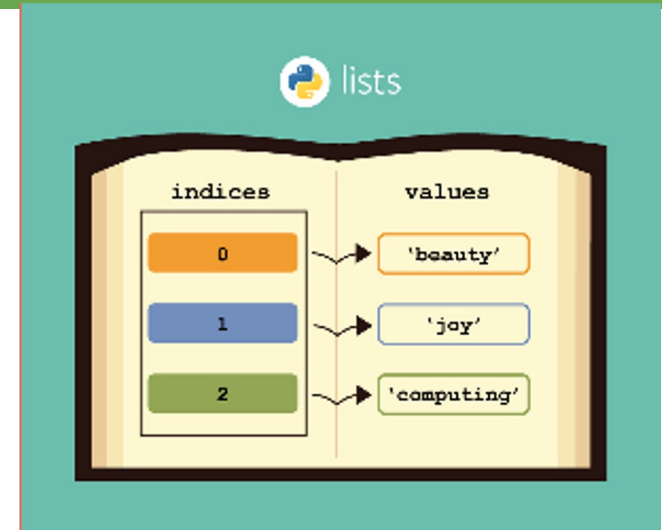
- Arithmetic: + - \* / % \*\* //
- Comparison: > < >= <= == !=
- Logical: and, or, not
- Assignment + arithmetic (some shortcuts): +=, -=, ...



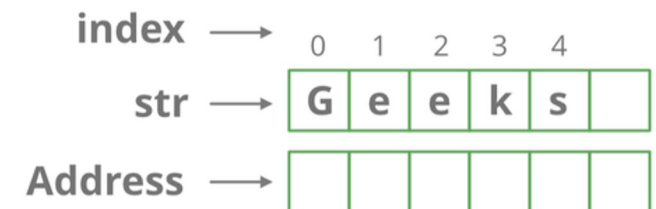
# Data Types for Sequences

Sequences: A sequences of numbers

- **Lists:** [1, 2, 3, 4, 5]
- **Tuples:** (1, 2, 3, 4, 5)
- **Strings:** "I love POLI3148"



Why is a string a *sequence*?



## Operators for Lists and Tuples (1)

- Access an element at a specified location: `x[0]`
- Manipulate an element at a specified location: `x[0] = 1 + 1`
  - Refers to “Numbers” for manipulation you can do
  - You can’t do it for **tuples**! Why?  
Tuples are **immutable** (no change after defined)

Note: Indices starts from ZERO (0), not ONE (1).

---

## Operators for Lists and Tuples (2)

- Check if a list or tuple contains an element  
`1 in [1, 2, 3]`                      `1 not in [1, 2, 3]`
  - Concatenate two sequences:  
`[1, 2, 3] + [4, 5]`
  - Add an element in the end of a **list**:  
`[1, 2, 3].append(4)`
-

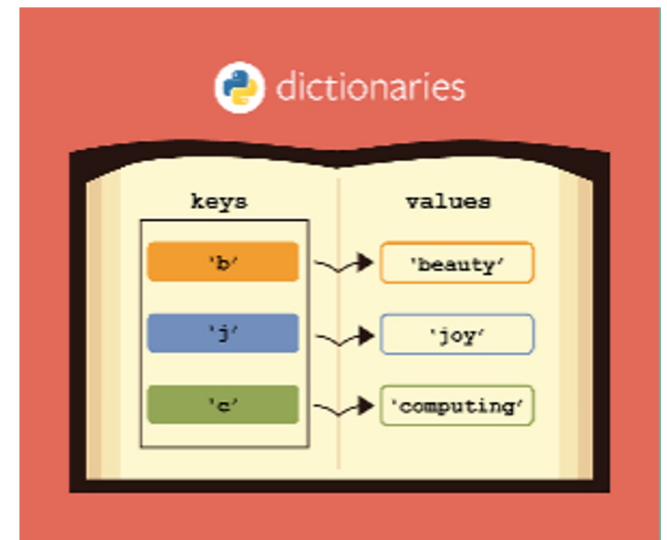
# Operators for Strings

- Identifiers of strings
  - One quotation mark for strings without line breaks
  - Three quotation marks for strings with line breaks
- Concatenate strings: +  
"I love" + "POLI3148"
- Format strings:  
  
course = "POLI3148"  
f"I love {course}"

# Data Type for Mapping

## Dictionary

- Store data in **key-value** pairs
- Ex:  
a = {"one": 1, "two": 2, "three": 3}  
b = {"China": "CHN", "United States": "USA"}



---

Check data types

`x = [1, 2, 3]`

`type(x)`

`type(x[0])`

---

# Advanced: Nested Structures

## A list of lists:

```
[["China", "USA"], [1, 1], [2, 2], [3, 3]]
```

## A list of dictionary

```
[{"China": 1, "USA": 1}, {"China": 2, "USA": 2}, {"China": 3, "USA": 3}]
```

## A dictionary of list

```
{"China": [1, 2, 3], "USA": [1, 2, 3]}
```

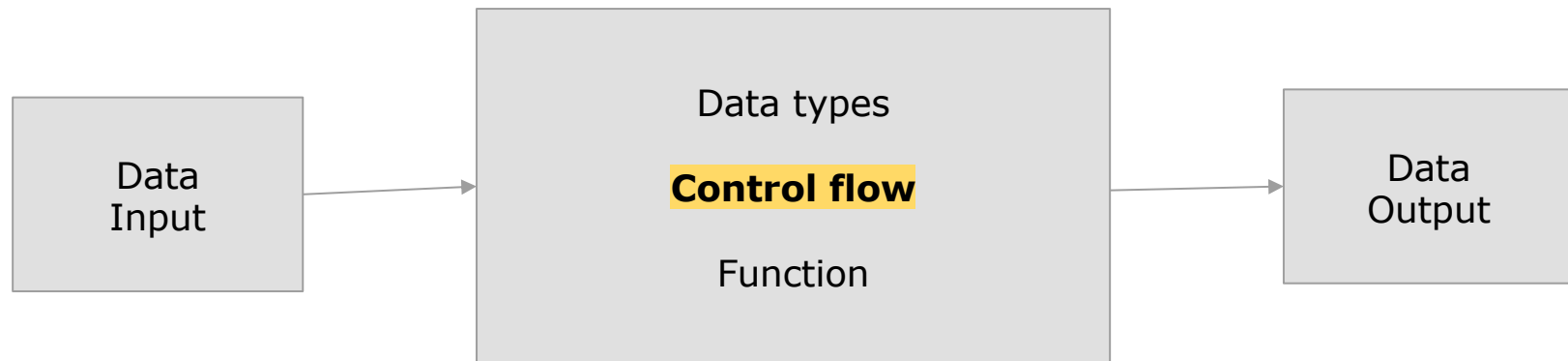
## A dictionary of dictionary

```
{"China": {"1": 1, "2": 2, "3": 3}, "USA": {"1": 1, "2": 2, "3": 3}}
```

Watch out for the different types of brackets “[ ]” and “{ }”

# Control flow

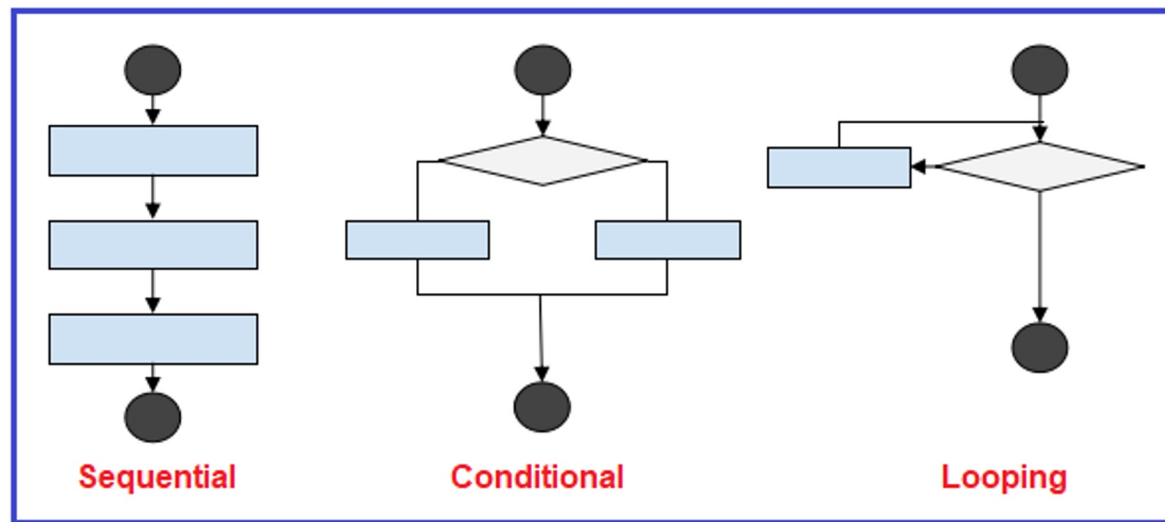
- Data Input
- Data processing
  - Data types
  - **Control flow**
  - Function
- Data Output





# What is a control flow?

Control flow (or flow of control) is the **order** in which individual statements, instructions, or function calls of an imperative program are executed or evaluated



---

# Control flow

## Statements

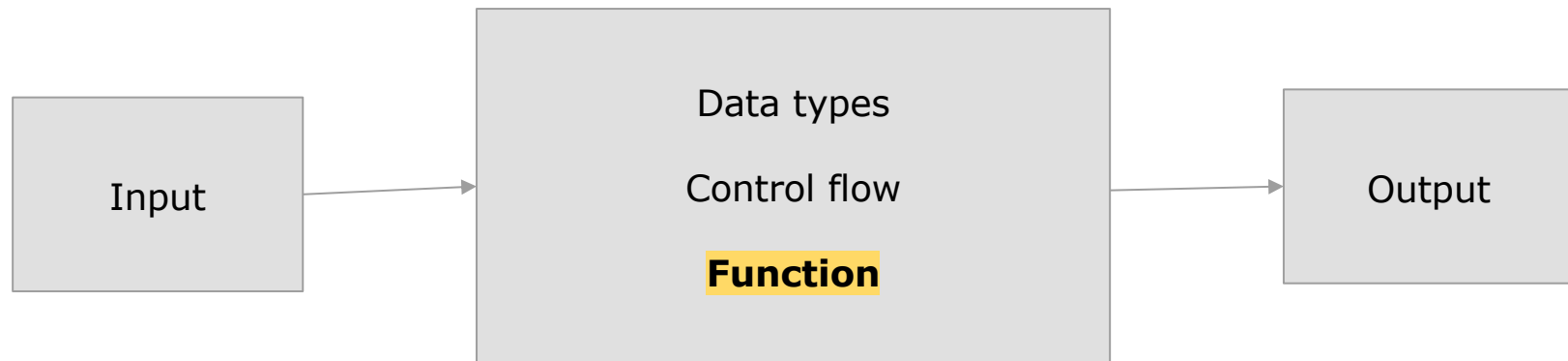
- **Condition:** if... elif... elif... else
- **Loop:** for

## Note:

- Define conditions with the **Boolean** data type
  - Define loops with the **list, tuple, and dictionary** data types
-

# Functions

- Data Input
- Data processing
  - Data types
  - Control flow
  - **Function**
- Analysis Output



# What is a function?

What is a function like?

- A function is a **block of code** which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

What is a function for?

A function **bundles** a set of instructions that you can use repeatedly.



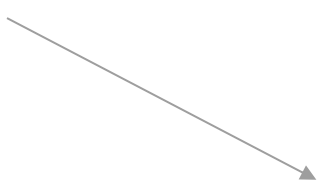
# Functions

- A function should have a name
- Objects inside a function are local
- A function can return data if you want  
(but you don't have to if you don't need them)

```
def calculate_addition(x, y):  
    z = x + y  
    print(z)  
    return(z)
```

## Further readings

- [Automate the boring stuff with Python](#): Chapters 1-6
- [The Python tutorial](#)
- [W3 School](#)



| Python Tutorial     |  |
|---------------------|--|
| Python HOME         |  |
| Python Intro        |  |
| Python Get Started  |  |
| Python Syntax       |  |
| Python Comments     |  |
| Python Variables    |  |
| Python Data Types   |  |
| Python Numbers      |  |
| Python Casting      |  |
| Python Strings      |  |
| Python Booleans     |  |
| Python Operators    |  |
| Python Lists        |  |
| Python Tuples       |  |
| Python Sets         |  |
| Python Dictionaries |  |
| Python If...Else    |  |
| Python While Loops  |  |
| Python For Loops    |  |
| Python Functions    |  |

---

## Further readings

- Python visualizer: [pythontutor.com/python-compiler.html](http://pythontutor.com/python-compiler.html)
  - Check out out Stream tab for more insights for peers' questions.
-

---

# In-Class Exercise 1 Answers and Feedback

---



## In-Class Exercise 1 Answers and Feedback

- Great job starting off with the exercises
- Debugging with Google
- Indentation
- = vs. ==
- isinstance( ) vs. type()
- Functions that return something vs. those that don't return (returns None)
- Pure functions vs. functions with side effects

---

## In-Class Exercise 2

---